



华中科技大学

操作系统原理课程设计报告

姓 名：郝依凝
学 院：计算机科学与技术学院
专 业：计算机科学与技术
班 级：CS2304
学 号：U202315653
指导教师：谢美意

分数	
教师签名	

2026 年 4 月 29 日

目 录

实验一 lab1_challenge2 打印异常代码行	1
1.1 实验目的	1
1.2 实验内容与实验记录	1
1.3 实验调试及心得	3
实验二 lab1_challenge3 多核启动及运行	4
2.1 实验目的	4
2.2 实验内容与实验记录	4
2.3 实验调试及心得	7
实验三 lab2_challenge2 堆空间管理	8
3.1 实验目的	8
3.2 实验内容与实验记录	8
3.3 实验调试及心得	10
实验四 lab2_challenge3 多核内存管理	11
4.1 实验目的	11
4.2 实验内容与实验记录	11
4.3 实验调试及心得	15
实验五 lab3_challenge2 实现信号量	16
5.1 实验目的	16
5.2 实验内容与实验记录	16
5.3 实验调试及心得	19
实验六 lab3_challenge3 写时复制	20
6.1 实验目的	20
6.2 实验内容与实验记录	20
6.3 实验调试及心得	25
实验七 lab4_challenge2 重载执行	26
7.1 实验目的	26
7.2 实验内容与实验记录	26
7.3 实验调试及心得	30
实验八 lab4_challenge3 简易 Shell	31
8.1 实验目的	31
8.2 实验内容与实验记录	31
8.3 实验调试及心得	36
实验九 challengeX : Put it all together	37

9.1 实验目的	37
9.2 实验内容与实验记录	37
9.3 实验调试及心得	47

实验一 lab1_challenge2 打印异常代码行

1.1 实验目的

本实验通过修改 PKE 内核，使得用户程序在发生异常时，内核能够输出触发异常的用户程序的源文件名和对应代码行，其中文件名需要包含路径，如果是用户源程序发生的错误，路径为相对路径，如果是调用的标准库内发生的错误，路径为绝对路径。

这个实验需要理解机器模式下的异常捕获与处理路径，掌握异常发生时程序计数器为核心寄存器的保存与提取机制，解析 ELF 文件的调试信息，并主要学习 `.debug_line` 段的数据组织结构，从而将触发异常的机器指令地址精准映射到高级语言源程序的文件路径与具体行号。本次实验中，我们通过综合运用异常上下文提取、DWARF 调试信息解析、地址与源代码行映射表构造等底层知识，极大地提升了在复杂系统环境下进行底层错误追踪与内核级问题定位的能力。实验不仅加深了我们对操作系统内核异常处理机制的理解，也为后续学习现代调试器核心机制、故障转储分析等高级软件诊断技术奠定了坚实的基础。

1.2 实验内容与实验记录

Part1.从 ELF 文件中提取并加载`.debug_line` 节数据

由于在 ELF 文件中 Section Header 里面并没有直接保存节的名字，而是保存偏移量，真正的名字字符串都集中存放在 `.shstrtab` 里。所以为获取 `.debug_line` 节数据，需要计算出字符串表节头位置，并根据节头中的信息将包含所有名字的字符串数据完整读取到缓冲区中。

获取到包含全部节名字的缓冲区即可遍历全部节进行比较，找到 `.debug_line` 节并保存它的节头，根据节头即可读取节数据。获取以上数据后即可调用系统中提供的 `make_addr_line` 函数进行解析以及映射。

最终在加载用户应用程序的函数 `load_bincode_from_host_elf` 中调用通过上述过程实现的从 ELF 文件中提取并加载 `.debug_line` 节数据函数 `load_debug_line` 实现对调试信息的加载，即可完成带有调试信息的完整用户程序文件的载入。

Part2.定位并提取错误内容

为实现在执行异常代码时能够输出错误代码行号、错误代码内容，最终提供

错误信息并退出程序，需要修改 `mtrap` 中非法指令的处理逻辑，即在处理非法指令函数处添加打印错误代码行号以及内容的处理逻辑。

首先需要通过读取机器模式异常程序计数器 `MEPC` 获取引发异常的那条汇编指令的物理地址；由于在解析 `.debug_line` 节过程中已经获取地址-行号映射表，所以当前可以直接遍历进程保存的此表，通过比对地址即可找到发生异常的行号信息和文件信息。获取到的文件信息可以根据目录名和文件名拼接出异常文件完整的相对或绝对路径。错误定位实现代码如下：

```
uint64 epc = read_csr(mepc);
addr_line *errorline = 0;
code_file *errorfile = 0;

for (int i = 0; i < current->line_ind; i++)
{
    if ((current->line + i)->addr == epc)
    {
        errorline = (current->line) + i;
        break;
    }
    if (i == current->line_ind)
    {
        panic("Fail to get errorline address.\n");
    }
}
errorfile = current->file + errorline->file;
```

已经获取异常文件路径后即可通过 `spike_file_open` 函数打开该异常文件，并将文件内容读入缓冲区。在上面的定位过程中已知出错行位于第 `n` 行，通过数换行符 `\n` 的个数可以跳过前面的 `n-1` 行，定位出错行的开头，通过下一个换行符定位出错行的结尾。最后关闭文件，并把提取到的这一行源代码输出到终端上。定位代码如下：

```
int code_start = 0;
for (int i = 0; i < errorline->line - 1; i++)
{
    while (file_content[code_start] != '\n')
    {
        code_start++;
    }
    code_start++;
}
```

Part3.实验测试结果

测试在用户态执行指令 `asm volatile("csrw sscratch, 0")` 触发异常，系统能够正确识别异常并定位异常代码位置。异常指令位于源代码第 14 行，测试结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_errorline
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_errorline
Application program entry point (virtual address): 0x0000000081000000
Switch to user mode...
Going to hack the system by running privilege instructions.
Runtime error at user/app_errorline.c:14
    asm volatile("csrw sscratch, 0");
Illegal instruction!
System is shutting down with exit code -1.
```

图 1 异常代码测试

本地调试 git 记录：

```
commit ffb52ee156eba27cd2eebe885388fd75fb2186c (HEAD -> lab1_challenge2_errorline, origin/lab1_challenge2_errorline)
Author: gabriel <3481963967@qq.com>
Date: Mon Feb 23 00:59:37 2026 +0800

    my work on lab1_challenge2 is done.

commit e2a3b2fd85ee7c6adbf9ae91928a290b4c4c3f39
Merge: d47e4b2 9d3384c
Author: gabriel <3481963967@qq.com>
Date: Wed Dec 24 15:52:58 2025 +0800

    continue to work on lab1_challenge1
```

图 2 lab1-2 git log

1.3 实验调试及心得

本次实验构建了操作系统内核底层对故障的追踪能力。当用户态程序执行非法指令如越权访问 `sscratch` 寄存器时，CPU 会触发陷阱。实验通过读取 `mepc` 精准捕获了触发异常的物理地址，利用 DWARF 调试信息格式中的 `.debug_line` 节，通过解析 ELF 文件，我们将这部分数据在内核中展开为内存地址与源代码文件以及行号的映射结构。当异常发生时，内核不只是 `panic` 停机，而是能够利用捕获到的 `mepc` 在映射表中反查，并在最后借助 Spike 模拟器的 HTIF 接口跨越宿主机与虚拟机的边界，读取并打印出引发崩溃的那一行具体源代码。

通过本次实验，我深刻体会到了为完成错误代码的识别与定位，底层需要完成从陷阱捕获与寄存器读取到符号表和行号表解析最后整合信息进行输出的一整套复杂过程。同时本次实验让我加深了对 ELF 格式的理解，为获取错误行信息，我深入接触了平时被忽略但至关重要的调试信息段 `.debug_line` 节，并学习了将提取到的 `.debug_line` 节转换为映射表的过程。

实验二 lab1_challenge3 多核启动及运行

2.1 实验目的

本次实验通过修改 PKE 内核，使其支持在 spike 模拟器上同时启动两个硬件线程，并分别在每个核上加载并运行独立的用户程序。实验需要深入理解多核系统的启动流程与并发执行机制，掌握 RISC-V 机器模式下的 hart 编号读取、核间同步与共享资源的并发控制方法。在此基础上，需要分离各核的内存地址空间，为每个核建立独立的时钟中断与进程上下文，并利用同步原语保护物理内存分配等临界区，最终在所有核执行完毕后协调关闭模拟器。

通过本实验，我们综合运用了多核启动同步、资源隔离、并发互斥与进程调度等底层知识，深刻理解了多处理器环境下操作系统内核的同步与调度机制，极大地提升了在复杂并行系统中进行内核级并发控制与系统性能调优的能力。实验不仅巩固了我们对操作系统内核多核架构的理解，也成为后续学习多线程并行编程、分布式系统一致性协议等高级计算机系统设计技术的基础。

2.2 实验内容与实验记录

Part1.PKE 代理内核初始化

在 M 模式下，minit 文件内完成每个 CPU 核心的底层初始化，并最终将控制权交给运行在 S 模式的操作系统内核。因此，需要修改该文件：进行唯一性资源初始化，即仅由 hart 0 负责初始化 HTIF 主机接口和获取模拟物理内存信息，其他 hart 在同步屏障处等待，保证所有 hart 都在 hart 0 完成上述初始化后才继续启动，实现多核同步。实现代码如下：

```
if (!hartid)
{
    spike_file_init();

    init_dtb(dtb);
}
sync_barrier(&counter, NCPU);

sprintf("In m_start, hartid:%d\n", hartid);
write_tp(hartid);
```

为实现多核的隔离，需要将寄存器保存帧 `g_itrframe` 修改为 `g_itrframe[NCPU]`，为多个内核分别分配 M 模式中断或异常处理时保存 CPU 上下文的内存区域。由于每个 hart 都有自己独立的定时器比较寄存器，必须各自初始化才能在自己的时间点上收到时钟中断，所以应对不同 hart 分别进行 `timerinit`。

Part2.S 模式的多核处理

在修改 M 模式后，根据内核启动流程，内核会进入 S 模式，因此接下来对 S 模式进行修改。为实现多核隔离，需要为每个 hart 分配一个独立的进程控制块，并初始化一个用户进程的上下文和地址空间。具体实现需要完成根据 hart 编号为不同核心计算独立的物理地址，避免多核间的内存冲突；将用户态的 `tp` 寄存器设为 `hartid`，使应用程序能识别自己运行在哪个核心上；调用 `load_bincode_from_host_elf`，根据命令行参数把对应 hart 的可执行文件加载到内存中，并填写进程入口地址。具体代码如下：

```
void load_user_program(process *proc)
{
    int tp = read_tp();

    proc->trapframe = (trapframe *)USER_TRAP_FRAME + 0x300000 * tp;
    memset(proc->trapframe, 0, sizeof(trapframe));

    proc->kstack = USER_KSTACK + 0x300000 * tp;
    proc->trapframe->regs.sp = USER_STACK + 0x300000 * tp;
    proc->trapframe->regs.tp = tp;

    // load_bincode_from_host_elf() is defined in kernel/elf.c
    load_bincode_from_host_elf(proc);
}
```

同时对于 `strap` 也需要进行修改。全局变量 `g_ticks` 用于记录每个 hart 经历的时钟中断次数，因此需要进行隔离，修改为 `g_ticks[NCPU]`。由于操作系统在 S 模式下的陷阱处理总入口函数 `smode_trap_handler` 需要对当前进程进行处理，因此需要修改进程管理文件，将当前进程管理块 `current` 拓展为数组 `current[NCPU]`，在对当前进程进行处理时，需要先调用函数 `read_tp` 获取当前 hart 编号，从而获取对应线程的进程。而在跳转到用户模式下运行指定的进程时，也需要将进程与线程进行绑定，保证进程与硬件线程的绑定关系从进程创建时就能够确立。

在退出系统时，S 模式下的 `sys_user_exit` 函数在任何一个核试图关机之前需要确保所有核上的用户进程都已经完成了退出操作，避免出现某个核还在运行而系统就被关闭的情况，因此需要在此处设置同步屏障。关机操作本身只需要执行

一次，而且由主核来执行符合常见的多核启动与退出逻辑，因此只让 hart 0 执行关机。具体实现代码如下：

```
ssize_t sys_user_exit(uint64 code)
{
    int tp = read_tp();
    sync_barrier(&excounter, NCPU);

    if (!tp)
        shutdown(code);

    return 0;
}
```

Part3.实验测试结果

测试 riscv-pke 通过 spike 启动两个 CPU，并且让 CPU0 执行 app0，让 CPU1 执行 app1。user0.lds 和 user1.lds 中规定了 app0 从 0x81000000 处开始加载，app1 从 0x81500000 处开始加载。测试结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike -p2 ./obj/riscv-pke ./obj/app0 ./obj/app1
HTIF is available!
(Emulated) memory size: 2048 MB
In m_start, hartid:0
hartid = 0: Enter supervisor mode...
hartid = 0: Application: ./obj/app0
hartid = 0: Application program entry point (virtual address): 0x0000000081000000
hartid = 0: Switch to user mode...
In m_start, hartid:1
hartid = 1: Enter supervisor mode...
hartid = 1: Application: ./obj/app1
hartid = 1: Application program entry point (virtual address): 0x0000000081500000
hartid = 1: Switch to user mode...
hartid = 0: >>> app0 is expected to be executed by hart0
hartid = 1: >>> app1 is expected to be executed by hart1
hartid = 0: User exit with code:0.
hartid = 1: User exit with code:0.
hartid = 0: shutdown with code:0.
System is shutting down with exit code 0.
```

图 3 多核测试

本地调试 git 记录：

```
commit 9651d38c161ee7cbda796db6eea82b9b57fa64d9 (HEAD -> lab1_challenge3_multicore, origin/lab1_challenge3_multicore)
Author: gabriel <3481963967@qq.com>
Date: Tue Feb 24 03:40:05 2026 +0800

    my work on lab1_challenge3 is done.

commit f6510a2c078937679019547ac20ed7927b14c805
Merge: 0fa9773 9d3384c
Author: gabriel <3481963967@qq.com>
Date: Mon Feb 23 01:19:17 2026 +0800

    continue to work on lab1_challenge3
```

图 4 lab1-2 git log

2.3 实验调试及心得

本次实验实现了 PKE 操作系统内核对多核并行运行的支持。当 spike 模拟器通过 -p2 参数同时启动两个硬件线程时，两个 hart 会从 _mentry 入口并发执行机器模式启动代码。在实际调试过程中，最棘手的问题之一在于 HTIF 接口与物理内存等共享资源的竞争。由于两个核同时调用 spike_file_init 和 init_dtb 会导致宿主机端的文件接口被重复初始化，进而引发后续的文件打开失败或内存信息丢失。解决该问题需要引入同步屏障 sync_barrier，只让 hart 0 执行这些唯一性初始化操作，并通过计数等待确保 hart 1 跳过这些步骤后才同步继续。另一个关键的问题是物理地址空间的分离。在单核实验中，进程使用固定宏定义的地址已经足够，但在多核环境下，两个核若共用这些地址会导致栈帧互相覆盖、定时器比较寄存器写入冲突。通过在 load_user_program 中为每个 hart 添加 $0x300000 * tp$ 的偏移量，为两个硬件线程划分了独立的陷阱帧、内核栈和用户栈区域，彻底隔离了内存空间。在 smode_trap_handler 中需要根据 read_tp 获取的 hart ID 来访问对应的 current[tp] 指针和 g_ticks[tp] 计数器，确保时钟中断和系统调用处理不会混淆不同核上的进程上下文。最终退出时，利用同步屏障让两个核都会聚到退出点，再由 hart 0 统一调用 shutdown，避免了一个核提前关闭系统导致另一个核崩溃的问题。

通过本次实验，我深刻体会到多核系统中并行执行与资源共享之间需要在细微之处精心权衡。多个核同时运行各自程序在底层需要解决从启动同步、内存布局隔离到中断分发、进程绑定以及退出协调的一系列复杂问题。实验中引入 tp 寄存器连接硬件线程与操作系统进程，每个 hart 通过 read_tp 能快速定位自己专属的数据结构和当前进程，感受到了 RISC-V 架构为多核支持提供的精巧硬件原语。同时，这次实验让我加深了对操作系统内核在并发场景下数据隔离原则的理解，任何一处共享数据的无保护访问都会导致系统出现难以复现的随机崩溃。这种从单核到多核并发的调试经历，加深了我对多线程并行编程的理解。

实验三 lab2_challenge2 堆空间管理

3.1 实验目的

本实验通过修改 PKE 内核，实现用户进程堆空间的精细化管理，使得 `better_malloc` 和 `better_free` 能够在一个物理页内按需分配和回收不同大小的内存块，确保申请紧凑、释放可重用。该实验需要深入理解进程虚拟地址空间布局、堆段扩展与回收机制、内存控制块的设计与维护、物理内存页的分配与释放流程，以及虚拟地址到物理地址的映射操作。

通过本次实验，我们综合运用了动态内存分配策略、空闲块链表管理、系统调用路径处理的底层知识，显著提升了操作系统内存管理子系统的设计与实现能力。实验不仅加深了对堆管理核心机制和虚拟内存映射机制的理解，也是后续学习现代操作系统中更复杂的内存分配器、碎片整理和高效存储管理技术的基础。

3.2 实验内容与实验记录

Part1.系统调用重写

`sys_user_allocate_page` 处理用户程序内存分配请求，添加参数 `n` 传入用户程序希望分配的字节数。修改函数逻辑，在函数内部调用 `user_better_malloc(n)`，该函数是内核中实际负责堆内存分配的实现，它会在进程的堆空间中分配一块不小于 `n` 字节的连续内存，并返回该内存块的起始虚拟地址 `va`，最终将 `va` 返回给上层系统调用处理逻辑，进而传递回用户程序，作为 `better_malloc` 的返回值。

`sys_user_free_page` 是处理用户程序释放内存请求的内核函数，参数 `va` 是用户程序要释放的内存块的起始虚拟地址。函数内部调用 `user_better_free(va)`，完成对指定虚拟地址对应内存块的回收。两个函数共同构成了用户态 `better_malloc` 和 `better_free` 对应的系统调用内核入口。

Part2.精细化分配实现逻辑

添加已分配块数据结构：

```
typedef struct malloc_t
{
    uint64 va;
    size_t size;
    struct malloc_t *next;
```

```

    struct malloc_t *prev;
} malloc;

```

在进程管理结构中添加已分配内存块链表、空闲块链表：

```

malloc *malloc_list;
malloc *malloc_free_list;

```

实现 `user_better_malloc`。首先尝试复用空闲块，从 `malloc_free_list` 中查找一个大小不小于传入参数 `size` 的空闲块：

若找到，则从空闲链表中摘除该节点，并检查其虚拟地址范围 $[va, va+size)$ 覆盖的所有物理页是否都已映射，防止物理页已被回收。如果复用的块比请求的 `size` 大，则将其切割为两部分，其中前部 $[va, va+size)$ 作为本次分配结果，后部 $[va+size, va+block_size)$ 作为一个新的空闲块重新插入 `malloc_free_list`。最后将该节点插入 `malloc_list`，返回 `va`；

若未找到可复用块，则从堆顶 `g_ufree_page` 分配新空间，之后需要确保从 `va` 开始的 `size` 字节范围内所有页面都已映射。最终将节点信息填入后插入 `malloc_list` 头部，更新 `g_ufree_page = va + size`，返回 `va`。

实现 `user_better_free`。首先在 `malloc_list` 中查找 `va` 对应的节点，找不到则直接返回。若找到则将该节点从 `malloc_list` 摘除，重新链接到 `malloc_free_list` 头部。遍历该块覆盖的所有物理页，对每一页检查是否有任何已分配块的范围与给定的物理页有交集，即检查该页是否还被其他已分配块部分占用。若没有其他分配，说明该物理页上全部空间都已被释放，则调用 `user_vm_unmap` 取消虚拟地址映射并释放该物理页回空闲物理内存池。若有其他分配，则保留映射，不做回收。这样，物理页只有在页内所有小块都释放后才会被回收，避免了过早释放导致其他块失效。

检查页是否可被释放代码如下：

```

static int page_has_other_allocation(uint64 page_va)
{
    for (malloc *m = current->malloc_list; m != NULL; m = m->next)
    {
        if (m->va + m->size > page_va && m->va < page_va + PGSIZE)
            return 1;
    }

    return 0;
}

```

Part3.实验测试结果

测试程序先利用 `better_malloc` 分别申请 100 和 50 个字节的物理页的内存，然后使用 `better_free` 释放掉 100 个字节，向 50 个字节中复制一串字符串，进行输出。程序第一次申请的两个内存应在同一物理页上，且第二次申请的内存起始地址应与第一次申请的 100 字节起始地址一致，测试结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_singlepageheap
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080008000, PKE kernel size: 0x0000000000008000 .
free physical memory address: [0x0000000080008000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_singlepageheap
Application program entry point (virtual address): 0x000000000001008a
Switch to user mode...
hello, world!!!
User exit with code:0.
System is shutting down with exit code 0.
```

图 5 精细化空间管理测试

本地调试 git 记录：

```
commit a3608ff6c006827c80c7a4747a0bf8301793cd41 (HEAD -> lab2_challenge2_singlepageheap, origin/lab2_challenge2_singlepageheap)
Author: gabriel <3481963967@qq.com>
Date: Thu Feb 26 05:08:53 2026 +0800

    my work on lab2_challenge2 is done.

commit ecedd6427733fa36f58ea3bbf0bc0adfee75f444
Merge: 8b0f6ef 5cae99a
Author: gabriel <3481963967@qq.com>
Date: Tue Feb 24 03:42:02 2026 +0800

    continue to work on lab2_challenge2
```

图 6 lab2-2 git log

3.3 实验调试及心得

本次实验构建了操作系统内核的进程堆空间精细化管理能力。通过在 `malloc_list` 和 `malloc_free_list` 两个链表上维护已分配与空闲内存块，实现了在一个物理页内按需切割、复用不同大小的内存区域，并可以在释放时自动判断物理页是否可以回收。调试过程中必须严格检查物理页内是否还有其他分配块，否则过早取消映射会导致残留指针访问已释放的物理内存，造成系统崩溃。

通过本实验，我深刻体会到 `malloc/free` 函数在内核中需要完成从虚拟地址空间管理、物理页分配映射、元数据节点维护到碎片合并与回收的一整套复杂机制。为了支持堆页内的紧凑分配，我深入理解了空闲链表的分裂与合并算法，以及如何利用进程的页表将离散的虚拟地址区间与物理内存动态挂接。实验让我认识到在资源受限的场景下，通过复用空闲节点、延迟物理页释放等设计可以显著提升内存利用率，为后续学习更高级的存储管理技术奠定了扎实的实践基础。

实验四 lab2_challenge3 多核内存管理

4.1 实验目的

本次实验与 lab1_challenge3 实验相同,通过修改 PKE 内核,使其支持在 spike 模拟器上同时启动两个硬件线程,并分别在每个核上加载并运行独立的用户程序。此外,本次实验需要完成多核环境下的内存分配与回收操作。实验需要深入理解多核系统中物理内存与虚拟地址空间的隔离机制,掌握基于自旋锁的互斥控制方法,确保各核在分配和释放物理页时不会产生冲突或重复映射。同时,需要为每个核维护独立的虚拟地址空间管理结构,保证各进程通过 `naive_malloc` 获得的虚拟地址连续且各自独立,并利用同步原语保护临界资源,最终在所有核执行完毕后协调退出。

通过本实验,我们综合运用了多核内存隔离、并发互斥、锁机制与虚拟地址空间管理等底层知识,深刻理解了多处理器环境下操作系统内核在内存资源分配与回收过程中的同步与隔离机制,极大地提升了在并行环境中进行内核级资源管理与并发控制的能力。

4.2 实验内容与实验记录

Part1.自旋锁机制

实验需要基于 RISC-V A 扩展实现了一个简单的自旋锁,用于多核环境下的临界区保护。首先对交换函数封装,用于自旋锁实现: `amoswap_d` 保证交换操作本身是原子的,但不限制编译器或 CPU 对这段代码前后的其他普通读写指令进行乱序优化; `amoswap_d_aq` 用于加锁操作,保证在这个原子操作之后的任何代码绝对不能跑到这个操作之前执行; `amoswap_d_rl` 用于解锁操作,保证在这个原子操作之前的所有代码必须全部真正写回到内存后才能执行这个交换操作,即确保当释放锁时其他 CPU 核看到的是修改后的最新数据。以 `amoswap_d` 为例封装代码如下:

```
static inline uint64 amoswap_d(volatile uint64 *ptr, uint64 value){
    uint64 result;
    asm volatile("amoswap.d %0, %2, (%1)" : "=r"(result) : "r"(ptr), "r"(value) :
"memory");
    return result;
}
```


基于上述交换函数实现自旋锁。设置锁状态，0 代表锁空闲，1 代表锁被占用；分别实现锁的初始化、死等模式的加锁、解锁、不等待模式的尝试加锁。注意，锁 lock 需要使用 volatile 关键字防止优化过程把 lock 的值缓存在寄存器里。具体实现代码如下：

```
#define SPIN_LOCK_FREE 0
#define SPIN_LOCK_HELD 1

typedef struct
{
    volatile uint64 lock;
} spinlock_amo_t;

static inline void spinlock_amo_init(spinlock_amo_t *lk)
{
    lk->lock = SPIN_LOCK_FREE;
}

static inline void spinlock_amo_lock(spinlock_amo_t *lk)
{
    while (amoswap_d_aq(&lk->lock, SPIN_LOCK_HELD) != SPIN_LOCK_FREE)
        ;
}

static inline void spinlock_amo_unlock(spinlock_amo_t *lk)
{
    amoswap_d_rl(&lk->lock, SPIN_LOCK_FREE);
}

static inline int spinlock_amo_trylock(spinlock_amo_t *lk)
{
    return amoswap_d_aq(&lk->lock, SPIN_LOCK_HELD) != SPIN_LOCK_FREE;
}
```

Part2.多核内存隔离

对于物理页分配，需要保证多核状态下不会分配到同一块物理地址，因此需要修改 pmm 文件。在分配前进行加锁，分配结束释放锁；同时为保证同一块物理地址不会被重复释放，在释放前后也需要分别进行加锁解锁操作；锁在初始化函数 pmm_init 中调用已经实现的 spinlock_amo_init 进行初始化即可。

物理内存管理器以及系统内核页表构建只需要进行一次，与 lab1_challenge3 相同，采用仅由 hart 0 负责初始化而其他 hart 在同步屏障处等待的方式，保证所

有 hart 都在 hart 0 完成上述初始化后才继续启动，实现多核同步。实现代码如下：

```
if (!tp)
{
    // init phisical memory manager
    pmm_init();

    // build the kernel page table
    kern_vm_init();
}
sync_barrier(&mem_counter, NCPU);
```

在多核环境下，每个核心可能并发执行不同的用户进程，它们各自的堆空间必须相互隔离，因此用 `g_ufree_page[NCPU]` 替代原来的 `g_ufree_page` 来独立管理当前核心的内存分配，每个核心都有自己独立的堆顶标记，互不干扰。最终实现在分配物理页时虚拟地址的正确维护：

```
uint64 sys_user_allocate_page()
{
    int tp = read_tp();
    void *pa = alloc_page();
    uint64 va = g_ufree_page[tp];
    g_ufree_page[tp] += PGSIZE;
    user_vm_map((pagetable_t)current[tp]->pagetable, va, PGSIZE, (uint64)pa,
                prot_to_type(PROT_WRITE | PROT_READ, 1));
    sprint("hartid = %d: vaddr 0x%x is mapped to paddr 0x%x\n", tp, va, pa);
    return va;
}
```

其余多核处理方式与 `lab1_challenge3` 一致即可完成本实验。

Part3.实验测试结果

实验测试给定两个程序，每个程序会通过 `naive_malloc` 申请一些内存页，在内存页开始处写入一个 `int` 并打印内存页的虚拟地址，最后每个进程会打印自己写入内存页的数，并通过 `naive_free` 释放申请的内存页。`app_alloc0.c` 会依次写入并输出 0, 1, 2, 3, 4; `app_alloc1.c` 会依次写入并输出 5, 6, 7, 8, 9。

测试结果应为两个程序同时执行，交替输出，且需要观察输出的申请到的地址，保证两个程序申请到的物理地址不同，确认正确实现存储的隔离机制。实验需要保证两个核之间不会相互影响，因此需要保证 `user0` 依次输出 0, 1, 2, 3, 4; `user1` 依次输出 5, 6, 7, 8, 9。实验需要确认由 `hart0` 启动终止系统，保证唯一性资源仅初始化一次，关机操作仅执行一次。测试结果如下：


```

● gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike -p2 obj/riscv-pke obj/app_alloc0 obj/app_alloc1
HTIF is available!
(Emulated) memory size: 2048 MB
In m_start, hartid:0
hartid = 0: Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008000a000, PKE kernel size: 0x000000000000a000 .
free physical memory address: [0x000000008000a000, 0x0000000087ffffff]
kernel memory manager is initializing ...
In m_start, hartid:1
hartid = 1: Enter supervisor mode...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080006000
hartid = 0: User application is loading.
hartid = 1: User application is loading.
hartid = 0: user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
hartid = 0: Application: obj/app_alloc0
hartid = 1: user frame 0x0000000087fb8000, user stack 0x000000007ffff000, user kstack 0x0000000087fb7000
hartid = 1: Application: obj/app_alloc1
hartid = 0: Application program entry point (virtual address): 0x000000000010078
hartid = 1: Application program entry point (virtual address): 0x000000000010078
hartid = 0: Switch to user mode...
hartid = 0: alloc page 0x87fa4000
hartid = 0: alloc page 0x87fa3000
hartid = 1: Switch to user mode...
hartid = 0: vaddr 0x00400000 is mapped to paddr 0x87fa4000
hartid = 1: alloc page 0x87fa2000
hartid = 1: alloc page 0x87fa1000
=== user alloc 0 @ vaddr 0x00400000
hartid = 1: vaddr 0x00400000 is mapped to paddr 0x87fa2000
hartid = 0: alloc page 0x87fa0000
hartid = 0: vaddr 0x00401000 is mapped to paddr 0x87fa0000
>>> user alloc 1 @ vaddr 0x00400000
=== user alloc 0 @ vaddr 0x00401000
hartid = 1: alloc page 0x87f9f000
hartid = 1: vaddr 0x00401000 is mapped to paddr 0x87f9f000
hartid = 0: alloc page 0x87f9e000
hartid = 0: vaddr 0x00402000 is mapped to paddr 0x87f9e000
>>> user alloc 1 @ vaddr 0x00401000
=== user alloc 0 @ vaddr 0x00402000
hartid = 1: alloc page 0x87f9d000
hartid = 1: vaddr 0x00402000 is mapped to paddr 0x87f9d000
hartid = 0: alloc page 0x87f9c000
hartid = 0: vaddr 0x00403000 is mapped to paddr 0x87f9c000
>>> user alloc 1 @ vaddr 0x00402000
=== user alloc 0 @ vaddr 0x00403000
hartid = 1: alloc page 0x87f9b000
hartid = 1: vaddr 0x00403000 is mapped to paddr 0x87f9b000
hartid = 0: alloc page 0x87f9a000
hartid = 0: vaddr 0x00404000 is mapped to paddr 0x87f9a000
>>> user alloc 1 @ vaddr 0x00403000
=== user alloc 0 @ vaddr 0x00404000
hartid = 1: alloc page 0x87f99000
hartid = 1: vaddr 0x00404000 is mapped to paddr 0x87f99000
=== user 0: 0
>>> user alloc 1 @ vaddr 0x00404000
=== user 0: 1
>>> user 1: 5
=== user 0: 2
>>> user 1: 6
=== user 0: 3
>>> user 1: 7
=== user 0: 4
>>> user 1: 8
hartid = 0: User exit with code:0.
>>> user 1: 9
hartid = 1: User exit with code:0.
hartid = 0: shutdown with code:0.
System is shutting down with exit code 0.

```

图 7 多核内存管理测试

本地调试 git 记录:

```

commit bd73fecb1fb259aada758a47c40e9a1b5391eee (HEAD -> lab2_challenge3_multicoremem, origin/lab2_challenge3_multicoremem)
Author: gabriel <3481963967@qq.com>
Date: Thu Feb 26 23:48:09 2026 +0800

    my work on lab2_challenge3 is done.

commit ba100e8dc43a96db83d63085252943aabeedaf47
Merge: 23ac789 5cae99a
Author: gabriel <3481963967@qq.com>
Date: Thu Feb 26 05:09:47 2026 +0800

    continue to work on lab2_challenge3

```

图 8 lab2-3 git log

4.3 实验调试及心得

本次实验构建了操作系统内核在多核环境下的内存管理能力。当两个硬件线程同时在 `spike` 模拟器上启动并各自加载用户程序后，两个进程会并发调用 `naive_malloc` 和 `naive_free` 操作堆空间。实验通过 RISC-V 的原子指令 `amoswap.d` 实现自旋锁，保护全局空闲物理页链表的分配与回收操作，确保同一时刻只有一个核能修改链表结构。同时，利用同步屏障 `sync_barrier` 协调两个核的启动顺序，保证物理内存管理器和内核页表仅被初始化一次且所有核就绪后才继续执行。在此基础上，将原本单核下的全局变量 `g_ufree_page` 扩展为 `g_ufree_page[NCPU]` 数组，使每个核维护独立的虚拟地址空间，避免不同进程的堆地址重叠。

调试过程中的主要困难在于并发竞态问题的复现与定位。初期未加锁时，两个核几乎同时调用 `alloc_page`，导致空闲链表节点被重复分配或指针被错误覆盖，随机引发物理页重复映射或链表损坏导致的无限循环的异常。通过在 `alloc_page` 和 `free_page` 插入 `spinlock_amo_lock/unlock`，并利用 `Spike` 的日志输出观察各核的执行交错解决上述问题。

通过本次实验，我深刻体会到了多核环境下内存管理远比单核复杂。为了构建正确的并发控制，我深入理解 RISC-V 原子指令的内存序语义如何保证临界区内外的访存顺序，以及自旋锁在多核系统中的实现原理。同时，本次实验让我认识到资源隔离的重要性，通过为每个核维护独立的堆指针和进程上下文，操作系统才能支撑多个用户程序的同时执行。实验加深了我对多核并发、同步原语和虚拟内存隔离等核心概念的理解，为后续并行编程等技术积累了宝贵的实践经验。

实验五 lab3_challenge2 实现信号量

5.1 实验目的

本实验通过修改 PKE 内核，使得操作系统能够支持用户态信号量机制，实现进程间的同步与互斥。实验需要理解信号量的数据结构与 PV 操作原语，掌握用户态与内核态之间的系统调用参数传递、返回值处理以及进程状态转换。在实现过程中，必须深入解析进程控制块中状态字段的管理逻辑，理清就绪队列与阻塞线程的调度关系，并正确设计信号量在内核中的分配、释放和等待唤醒流程，从而将应用层的信号量操作精准映射为底层的内核资源管理行为。

实验需要添加系统调用，使得用户对信号量的操作可以在内核态处理。实验最终那个在内核中实现信号量的分配、释放和 PV 操作，当 P 操作处于等待状态时能够触发进程调度。

5.2 实验内容与实验记录

Part1.添加对信号量分配与操作的支持

在用户函数库 user_lib.h 加入 sem_new、sem_P、sem_V 函数的声明，并在对应的 user_lib.c 文件中加入对应的实现，从而进入系统调用，实现代码如下：

```
int sem_new(int init_val)
{
    return do_user_call(SYS_user_semNew, init_val, 0, 0, 0, 0, 0, 0);
}
void sem_P(int sid)
{
    do_user_call(SYS_user_semP, sid, 0, 0, 0, 0, 0, 0);
}
void sem_V(int sid)
{
    do_user_call(SYS_user_semV, sid, 0, 0, 0, 0, 0, 0);
}
```

在系统调用中加入对上述三个功能的支持。信号量的本质是进程间的同步与互斥，因此具体实现在 process 文件中进行，在系统调用中进行识别函数并传入参数即可。

Part2.信号量分配与操作的实现

设计信号量数据结构：

```
typedef struct semaphore_t {
    int ifuse;           // 分配标志
    int value;           // 信号量值
    process *wait_queue; // 等待队列
} semaphore;
```

采用数组结构记录信号量 `semaphore sem_table[NSEM]`，在进程初始化中加入对信号量数组的初始化。实现 `do_semNew` 分配信号量函数只需要遍历信号量数组，找到第一个未分配的信号量进行分配即可，实现代码如下：

```
int do_semNew(uint64 init_val)
{
    for (int i = 0; i < NSEM; i++)
    {
        if (sem_table[i].ifuse == 0)
        {
            sem_table[i].ifuse = 1;
            sem_table[i].value = (int)init_val;
            sem_table[i].wait_queue = NULL;
            return i;
        }
    }
    return -1;
}
```

实现 P 操作函数 `do_semP` 需要先对信号量 `id` 做合法性检查。若该信号量 `value > 0`，表示有空闲资源，`value` 数值减一后返回，进程继续执行。若 `value` 值为 0，则当前进程必须等待，即将 `current` 加入到该信号量的等待队列末尾，进程状态置为 `BLOCKED`，然后调用 `schedule` 切换到其他就绪进程。实现代码如下：

```
void do_semP(uint64 sid)
{
    if (sid >= (uint64)NSEM || sem_table[sid].ifuse == 0)
    {
        panic("do_semP: invalid sid\n");
        return;
    }
    semaphore *sem = &sem_table[sid];
    if (sem->value > 0)
    {
        sem->value--;
        return;
    }
}
```

```

    }

    current->queue_next = NULL;
    if (sem->wait_queue == NULL)
    {
        sem->wait_queue = current;
    }
    else
    {
        process *p = sem->wait_queue;
        while (p->queue_next != NULL)
            p = p->queue_next;
        p->queue_next = current;
    }
    current->status = BLOCKED;
    schedule();
}

```

实现 V 操作函数 `do_semV` 同样需要先对信号量 `id` 做合法性检查。通过合法性检查后首先对 `value` 值加一，表示释放了一个资源。若等待队列非空，则唤醒一个阻塞的进程，即从等待队列头部取出一个进程，将 `value` 值减回去，因为被唤醒的进程会直接消耗掉刚释放的这个资源，所以总资源数并未增加。将该进程从等待队列摘除，并调用 `insert_to_ready_queue` 函数改变进程状态并将进程插入就绪队列，准备被调度。若等待队列为空，则 `value` 保持+1 后的值，表示增加了一个空闲资源供后续 P 操作使用。实现代码如下：

```

void do_semV(uint64 sid)
{
    if (sid >= (uint64)NSEM || sem_table[sid].ifuse == 0){
        panic("do_semV: invalid sid\n");
        return;
    }
    semaphore *sem = &sem_table[sid];
    sem->value++;
    if (sem->wait_queue != NULL){
        process *p = sem->wait_queue;
        sem->wait_queue = p->queue_next;
        p->queue_next = NULL;
        sem->value--;
        insert_to_ready_queue(p);
    }
}

```

Part3.实验测试结果

测试程序通过信号量的增减，控制主进程和两个子进程的输出按主进程，第一个子进程，第二个子进程，主进程，第一个子进程，第二个子进程……这样的顺序轮流输出同。输出应为分别执行进程 0、进程 1、进程 2，打印相同数值。测试结果与预期相符，部分测试结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_semaphore
Child1 print 7
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent print 8
going to insert process 1 to ready queue.
going to schedule process 1 to run.
Child0 print 8
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Child1 print 8
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent print 9
going to insert process 1 to ready queue.
User exit with code:0.
going to schedule process 1 to run.
Child0 print 9
going to insert process 2 to ready queue.
User exit with code:0.
going to schedule process 2 to run.
Child1 print 9
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

图 9 信号量实现测试

本地调试 git 记录：

```
commit e8e809d68881c896362a1f47074d650da89fac37 (HEAD -> lab3_challenge2_semaphore, origin/lab3_challenge2_semaphore)
Author: gabriel <3481963967@qq.com>
Date: Fri Feb 27 01:03:40 2026 +0800

    my work on lab3_challenge2 is done.

commit 46ae4c45fdb4ff748c4a97aec1ec384777f303b1
Merge: 06f549a 788ffba
Author: gabriel <3481963967@qq.com>
Date: Thu Feb 26 23:49:36 2026 +0800

    continue to work on lab3_challenge1
```

图 10 lab3-2 git log

5.3 实验调试及心得

实验构建了操作系统内核对进程间同步与互斥的支撑能力。当用户程序创建信号量并执行 PV 操作时，内核通过信号量计数器与进程等待队列，动态控制进程的阻塞与唤醒，从而实现了多进程协调有序的运行。实验通过在系统调用中解析信号量操作语义，切换当前进程的状态并维护等待队列与就绪队列，重新进行调度，保证了进程执行流的安全切换。实验中我深刻体会到信号量底层机制对进程状态管理的精细要求，PV 操作需要完成资源检查、队列维护、状态跃迁和主动调度这一连串原子性协同动作。

实验六 lab3_challenge3 写时复制

6.1 实验目的

完成实验需要在堆段的复制发生时，实现先映射但不实际复制。同时需要正确设置权限，即当子进程拥有映射到父进程的地址的页表项时页表项不具有写权限，当 COW 发生后页表项可写。完成实验需要利用 PTE 的标志位中的两位 RSW 使内核能够分辨现在是否为 copy-on-write fork 场景。当父进程 fork 多个子进程后，一个物理页可能被多个进程的页表映射，该物理页的释放不能仅依赖于某一个进程的退出，实验必须确保没有任何进程还在使用它。

本实验通过修改 PKE 内核，使得用户程序在调用 fork 创建子进程时，子进程的堆内存页面不会立即复制，而是先与父进程共享同一物理页，仅当子进程尝试写入这些共享页面时才触发写时复制操作，从而推迟并减少不必要的内存拷贝。实验需要理解 RISC-V 页表项中标志位的语义与用途，掌握缺页异常的处理路径，并能够在异常处理函数中区分写时复制场景和常规缺页，完成物理页分配、内容拷贝及页表权限更新。

6.2 实验内容与实验记录

Part1.COW 识别与构建

在 RISC-V Sv39 页表项中，第 8 位和第 9 位属于 RSW 字段记录，硬件不会解释这两位，操作系统可以自由使用，因此本实验选取第一个 RSW 位作为写时复制标记，定义如下：

```
#define PTE_RSW (1L << 8)
#define PTE_COW PTE_RSW
```

由于顶层封装为虚拟地址，将 COW 页面的建立、标记与识别函数封装在虚拟地址管理的 vmm 文件中。实现将一个已有的、可写的虚拟页降级为只读且带 COW 标记的共享页，用于在 fork 时将父进程的页面变为 COW 状态，代码如下：

```
void user_vm_set_cow(pagetable_t page_dir, uint64 va){
    pte_t *pte = page_walk(page_dir, va, 0);
    if (pte == 0 || (*pte & PTE_V) == 0) return;
    *pte = (*pte & ~PTE_W) | PTE_COW;
}
```


实现 COW 识别函数 `user_vm_get_pte`，获取指定虚拟地址对应的 PTE 指针，用于检查 PTE 的权限或修改其内容，直接调用 `page_walk` 函数即可。实现映射函数，为子进程建立只读的、带 COW 标记的虚拟地址映射，将 `va` 起始的 `size` 大小区域映射到物理地址 `pa`，但不拷贝实际内容，从而实现“先映射不复制”。为实现上述功能，需要计算映射的起始页和结束页从而按页循环，对每一页调用 `page_walk`，若中间页表不存在则自动分配，同时需要检查该虚拟页是否已被映射，若已存在有效 PTE 则 `panic`，防止意外覆盖。为实现 COW 需要构造 PTE：`perm = PTE_R | PTE_A | PTE_U | PTE_COW | PTE_V`，PTE 中没有 `PTE_W` 从而保证写入会引发缺页。最终通过 `PA2PTE` 将物理地址转换为 PTE 物理页号部分再或上 `perm`，写入 PTE。实现代码如下：

```
void user_vm_map_cow(pagetable_t page_dir, uint64 va, uint64 size, uint64 pa)
{
    uint64 first, last;
    pte_t *pte;
    uint64 perm = PTE_R | PTE_A | PTE_U | PTE_COW | PTE_V; // read-only, COW
    for (first = ROUNDDOWN(va, PGSIZE), last = ROUNDDOWN(va + size - 1,
        PGSIZE); first <= last; first += PGSIZE, pa += PGSIZE)
    {
        if ((pte = page_walk(page_dir, first, 1)) == 0)
            return;
        if (*pte & PTE_V)
            panic("user_vm_map_cow: va 0x%lx already mapped", first);
        *pte = PA2PTE(pa) | perm;
    }
}
```

Part2.COW 物理页处理

为保证物理页的正确分配与释放，需要对物理页管理文件 `pmm` 进行修改。首先设置 `page_ref_count[PMM_MAX_PAGES]` 数组用于记录当前物理页的被引用个数，只有不再有进程使用该物理也时才能释放，进而保证 COW 模式下的安全物理页释放。即 `free_page` 中部分逻辑如下：

```
if (page_ref_count[idx] > 0)
    page_ref_count[idx]--;
if (page_ref_count[idx] == 0){
    list_node *n = (list_node *)pa;
    n->next = g_free_mem_list.next;
    g_free_mem_list.next = n;}
}
```

在分配物理页以及 fork 过程中需要正确维护物理页引用计数。在分配新物理页时，若成功分配则将引用计数设置为 1，在后续的 fork 与释放中对该计数进行增减。由于需要频繁增减以及获取指定物理地址的物理页引用计数，需要封装相应的函数，注意到减少引用计数逻辑与释放物理页逻辑一致，直接复用即可，最终实现代码如下：

```
void inc_page_ref(void *pa)
{
    if ((uint64)pa < free_mem_start_addr || (uint64)pa >= free_mem_end_addr)
        return;
    uint64 idx = ((uint64)pa - free_mem_start_addr) / PGSIZE;
    uint64 max_pages = (free_mem_end_addr - free_mem_start_addr) / PGSIZE;
    if (idx < max_pages)
        page_ref_count[idx]++;
}

void dec_page_ref(void *pa)
{
    free_page(pa);
}

int get_page_ref(void *pa)
{
    if ((uint64)pa < free_mem_start_addr || (uint64)pa >= free_mem_end_addr)
        return 0;
    uint64 idx = ((uint64)pa - free_mem_start_addr) / PGSIZE;
    uint64 max_pages = (free_mem_end_addr - free_mem_start_addr) / PGSIZE;
    return (idx < max_pages) ? page_ref_count[idx] : 0;
}
```

Part3.fork 修改

为实现 COW，需要修改 fork 逻辑，让父子进程共享同一物理内存但均只读，任何一方写入时才触发实际复制。由于栈是各进程私有的，STACK_SEGMENT 段直接分配新页并进行完整复制而保持独立；同理 CONTEXT_SEGMENT 段的 trapframe 直接赋值拷贝。代码段 CODE_SEGMENT 本就只读，因此只需让子进程映射到同一物理页并赋予 PROT_READ | PROT_EXEC 权限，无需 COW。最终需要重点处理堆段 HEAP_SEGMENT，堆段不再使用直接复制的方式增大开销，而是利用上述已经实现的构建 COW 页的函数将子进程逻辑地址与父进程物理地址进行映射，去掉父进程页表写权限，增加父进程物理页引用计数。实现代码如下：

```

// void* child_pa = alloc_page();
// memcpy(child_pa, (void*)lookup_pa(parent->pagetable, heap_block), PGSIZE);
// user_vm_map((pagetable_t)child->pagetable, heap_block, PGSIZE, (uint64)child_pa,
//             prot_to_type(PROT_WRITE | PROT_READ, 1));

// COW 修改后
uint64 pa = lookup_pa(parent->pagetable, heap_block);
if (pa == 0)
    continue;
user_vm_map_cow((pagetable_t)child->pagetable, heap_block, PGSIZE, pa);
user_vm_set_cow((pagetable_t)parent->pagetable, heap_block);
inc_page_ref((void *)pa);

```

Part4.strap 修改

通过上述 PTE 设置，当需要进行写时会触发缺页异常。触发缺页异常原因有多种，需要调用上述过程实现的 `user_vm_get_pte` 函数获取该页的 PTE，检查 `PTE_V` 和 `PTE_COW` 从而识别是否为 COW 引起的缺页异常。若不是，直接进行普通缺页处理，即分配新页并进行逻辑地址与物理地址映射即可。若是，需要检查当前页引用计数，引用计数 > 1 说明此时多个进程共享该页，需要分配一个新的物理页并将原物理页的内容完整拷贝到新页；需要修改当前进程的 PTE 而使其指向新物理页，并恢复读、写、用户权限，保持 `PTE_V`；需要调用上述实现的函数 `dec_page_ref` 减少原物理页的引用计数，表示当前进程不再共享它而其他只读进程仍共享原物理页，它们的引用计数不变，直到各自触发 COW 或退出。如果引用计数为 1，表示只有当前进程持有该页，此时无需分配新页，直接修改 PTE，即添加 `PTE_W` 写权限、`PTE_D` 脏位，清除 `PTE_COW` 标记。最后调用 `flush_tlb` 刷新 TLB，确保硬件使用更新后的页表项。实现代码如下：

```

uint64 va_page = read_csr(stval) & ~(PGSIZE - 1);
pte_t *pte = user_vm_get_pte((pagetable_t)current->pagetable, va_page);
if (pte && (*pte & PTE_V) && (*pte & PTE_COW)){
    sprint("handle_page_fault: %lx\n", read_csr(stval));
    uint64 pa = PTE2PA(*pte);
    if (get_page_ref((void *)pa) > 1){
        void *new_pa = alloc_page();
        memcpy(new_pa, (void *)pa, PGSIZE);
        *pte = PA2PTE((uint64)new_pa) | prot_to_type(PROT_READ | PROT_WRITE, 1)
| PTE_V;
        dec_page_ref((void *)pa);
    }
    else *pte = (*pte | PTE_W | PTE_D) & ~PTE_COW;
}

```

```

flush_tlb();

else{
    void *pa = alloc_page();
    user_vm_map((pagetable_t)current->pagetable, va_page, PGSIZE, (uint64)pa,
                prot_to_type(PROT_WRITE | PROT_READ, 1));
}

```

Part5.实验测试结果

的测试程序在父进程的堆上申请一片区域，并输出其物理地址，之后进行fork操作，输出子进程在对堆数据写入前后对应的物理地址。测试结果子进程在对堆数据写入前后的物理地址应不同，测试结果与预期相符，结果如下：

```

● gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike obj/riscv-pke obj/app_cow
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008002b000, PKE kernel size: 0x00000000002b000 .
free physical memory address: [0x000000008002b000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
Switch to user mode...
in alloc_proc. user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
User application is loading.
Application: obj/app_cow
CODE_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x000000000010078
going to insert process 0 to ready queue.
going to schedule process 0 to run.
the physical address of parent process heap is: 0000000087faf000
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087fad000, user stack 0x000000007ffff000, user kstack 0x0000000087fac000
do_fork map code segment at pa:0000000087fb2000 of parent to child at va:000000000010000.
going to insert process 1 to ready queue.
User exit with code:0.
going to schedule process 1 to run.
the physical address of child process heap before copy on write is: 0000000087faf000
handle_page_fault: 0000000000400000
the physical address of child process heap after copy on write is: 0000000087fa0000
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.

```

图 11 COW 测试

本地调试 git 记录：

```

commit a4360f1c19ccc8d4e497dfdb6d87f0d9ac46532d (HEAD -> lab3_challenge3_cow, origin/lab3_challenge3_cow)
Author: gabriel <3481963967@qq.com>
Date: Sun Mar 1 14:56:14 2026 +0800

    my work on lab3_challenge3 is done.

commit f2c39df22cd635d436ea0e6ece365d257f6df3ff
Merge: 452a824 788ffba
Author: gabriel <3481963967@qq.com>
Date: Fri Feb 27 01:05:51 2026 +0800

    continue to work on lab3_challenge3

```

图 12 lab3-3 git log

6.3 实验调试及心得

本次实验构建了操作系统内核中基于写时复制的内存管理优化能力。当父进程调用 `fork` 创建子进程时，堆空间页面不会立即拷贝，而是将父子进程的页表项同时标记为只读并设置 COW 标志，共享同一物理页。任何一方后续写入时触发 Store Page Fault，内核通过捕获 `stval` 获取缺页地址，结合页表中 RSW 保留位识别 COW 场景，根据引用计数决定是分配新页拷贝内容还是直接恢复写权限，从而延迟并减少物理内存的分配与拷贝开销。

通过本次实验，我深刻体会到了虚拟内存存在性能优化中的精巧设计，COW 的实现需要页表标记、缺页异常分发、物理页引用计数、TLB 刷新多模块紧密协作。调试过程中，存在解除映射后未同步刷新 TLB 的问题，造成进程读到过期数据。本次实验让我加深了对 Sv39 页表项格式的掌握，真正使用了保留位实现自定义软件语义，深刻理解了在 `fork` 密集型负载下 COW 是如何显著降低内存占用与复制延迟的。这些底层细节的实践，为我理解现代操作系统中的进程快照、轻量级容器以及页面替换策略提供了基础。

实验七 lab4_challenge2 重载执行

7.1 实验目的

本次实验采用在不修改 PKE 原有主程序加载方式即仍然通过命令行直接指定主程序映像的基础上，实现 `exec` 系统调用。实验核心目标是使一个用户程序能够在运行时动态加载，并执行另一个存放在文件系统中的用户程序。

实验需要掌握进程地址空间即代码段、数据段、堆栈的拆卸与重建过程，并能够完整替换当前进程的页表映射、堆管理结构以及用户态上下文。本次实验可以加深对进程程序映像动态替换机制的理解，为后续实现命令解释器等系统组件提供支持。

7.2 实验内容与实验记录

Part1.添加对重载执行的支持

在用户函数库 `user_lib.h` 加入 `exec` 函数的声明，并在对应的 `user_lib.c` 文件中加入对应的实现，从而进入系统调用，实现代码如下：

```
int exec(const char *pathname)
{
    return do_user_call(SYS_user_exec, (uint64)pathname, 0, 0, 0, 0, 0, 0);
}
```

在系统调用中加入对上述 `exec` 功能的支持。`exec` 的本质是替换当前进程，不是创建新进程，而是用新程序完全覆盖当前进程的地址空间和执行上下文，因此具体实现在 `process` 文件中进行，但在调用函数时传入地址为虚拟地址，所以在系统调用中进行识别函数并将可执行文件虚拟地址转换为物理地址传入即可。实现代码如下：

```
ssize_t sys_user_exec(char *pathva)
{
    char *pathpa = (char *)user_va_to_pa((pagetable_t)(current->pagetable),
        (void *)pathva);
    return do_exec(pathpa);
}
```

Part2.重载执行实现

在 process.h 文件中声明函数 `ssize_t do_exec(char *pathpa)`; 其中传入参数为新可执行文件的路径字符串，地址已转换为物理地址。首先打开目标文件并进行校验，校验通过后开始移除旧数据。遍历进程所有已映射区域，对代码、数据、栈、堆段逐一解除映射并释放物理页；将代码段和数据段的 `mapped_info` 记录清空，表示这些槽位可被新段使用；移除堆段记录，即找到堆段的记录项，清空其内容，并将 `total_mapped_region` 截断到该位置，此时 `mapped_info` 中只剩下系统保留段。实现代码如下：

```
// Unmap
for (int i = 0; i < proc->total_mapped_region; i++){
    int st = proc->mapped_info[i].seg_type;
    if (st == CODE_SEGMENT || st == DATA_SEGMENT ||
        st == STACK_SEGMENT || st == HEAP_SEGMENT){
        uint64 va = proc->mapped_info[i].va;
        uint32 npages = proc->mapped_info[i].npages;
        for (uint32 k = 0; k < npages; k++)
            user_vm_unmap(pagetable, va + k * PGSIZE, PGSIZE, 1);
    }
}

// Clear
for (int i = 0; i < proc->total_mapped_region; i++){
    int st = proc->mapped_info[i].seg_type;
    if (st == CODE_SEGMENT || st == DATA_SEGMENT){
        proc->mapped_info[i].va = 0;
        proc->mapped_info[i].npages = 0;
    }
}

{
    int i;
    for (i = 0; i < proc->total_mapped_region; i++){
        if (proc->mapped_info[i].seg_type == HEAP_SEGMENT){
            proc->mapped_info[i].va = 0;
            proc->mapped_info[i].npages = 0;
            proc->total_mapped_region = i;
            break;
        }
    }
}
```


清除旧数据后需要重置数据从而准备载入新程序。需要重建用户栈，为新程序分配一个物理页作为用户栈，映射到虚拟地址 `USER_STACK_TOP - PGSIZE`，在 `mapped_info` 中找到栈段记录，更新其虚拟地址和页数。需要重置堆管理器，将堆的起始和结束指针复位到 `USER_FREE_ADDRESS_START`，并清空已释放页列表，使堆回到初始空状态。实现代码如下：

```
// Re-setup stack
uint64 user_stack_pa = (uint64)alloc_page();
memset((void *)user_stack_pa, 0, PGSIZE);
user_vm_map(pagetable, USER_STACK_TOP - PGSIZE, PGSIZE, user_stack_pa,
            prot_to_type(PROT_READ | PROT_WRITE, 1));
for (int i = 0; i < proc->total_mapped_region; i++)
{
    if (proc->mapped_info[i].seg_type == STACK_SEGMENT)
    {
        proc->mapped_info[i].va = USER_STACK_TOP - PGSIZE;
        proc->mapped_info[i].npages = 1;
        break;
    }
}

// Reset heap
proc->user_heap.heap_top = USER_FREE_ADDRESS_START;
proc->user_heap.heap_bottom = USER_FREE_ADDRESS_START;
proc->user_heap.free_pages_count = 0;
```

经过上述清除与重构，可以加载新程序。首先加载新 ELF 的程序段，遍历 ELF 程序头表，仅加载类型为 `PT_LOAD` 的段，每个 `LOAD` 段分配一个物理页，建立映射，并将文件内容读入，将段的虚拟地址、大小、类型记录到 `mapped_info` 中，同时递增 `total_mapped_region`。加载完成后需要注册新的堆段，在 `mapped_info` 的空闲位置添加一个空的堆段记录，初始化起始地址，初始化页数为 0，等待后续动态分配。最终需要设置新进程入口与栈指针，将新 ELF 的入口地址写入 `trapframe->epc`，用户栈顶写入 `trapframe->regs.sp`，当系统调用返回，`switch_to` 会加载此 `trapframe`，使 CPU 从新程序入口开始执行。关键代码如下：

```
for (int i = 0, off = ehdr.phoff; i < ehdr.phnum; i++, off += sizeof(phdr)){
    void *pa = alloc_page();
    memset(pa, 0, PGSIZE);
    uint64 perm = prot_to_type(PROT_READ | PROT_WRITE | PROT_EXEC, 1);

    if (phdr.flags == (SEGMENT_READABLE | SEGMENT_EXECUTABLE))
        perm = prot_to_type(PROT_READ | PROT_EXEC, 1);
```

```

else if (phdr.flags == (SEGMENT_READABLE | SEGMENT_WRITABLE))
    perm = prot_to_type(PROT_READ | PROT_WRITE, 1);
user_vm_map(pagetable, phdr.vaddr, PGSIZE, (uint64)pa, perm);
int j;
for (j = 0; j < PGSIZE / sizeof(mapped_region); j++)
    if (proc->mapped_info[j].va == 0) break;
proc->mapped_info[j].va = phdr.vaddr;
proc->mapped_info[j].npages = 1;
if (phdr.flags == (SEGMENT_READABLE | SEGMENT_EXECUTABLE)){
    proc->mapped_info[j].seg_type = CODE_SEGMENT;
}
else if (phdr.flags == (SEGMENT_READABLE | SEGMENT_WRITABLE)){
    proc->mapped_info[j].seg_type = DATA_SEGMENT;
}
proc->total_mapped_region++;
}

```

Part3.实验测试结果

测试程序为 `app_exec` 和 `app_ls`。其中 `app_exec` 作为主程序，会在 PKE 启动后被运行，`user/app_ls.c` 被编译成可执行文件 `app_ls`，保存在 `./hostfs_root/bin/` 目录下。在 `app_exec` 执行过程中会调用 `exec` 系统调用执行 `/bin` 目录下的 `app_ls` 程序。测试结果如下：

```

● gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_exec
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080011000, PKE kernel size: 0x0000000000011000 .
free physical memory address: [0x0000000080011000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080009000
kernel page table is on
RAMDISK0: base address of RAMDISK0 is: 0x0000000087f35000
RFS: format RAMDISK0 done!
Switch to user mode...
in alloc_proc. user frame 0x0000000087f29000, user stack 0x000000007ffff000, user kstack 0x0000000087f28000
FS: created a file management struct for a process.
in alloc_proc. build proc_file_management successfully.
User application is loading.
Application: ./obj/app_exec
CODE_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x0000000000010078
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== exec /bin/app_ls in app_exec =====
Application: /bin/app_ls
CODE_SEGMENT added at mapped info offset:3
DATA_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x00000000000100b0
-----
ls "/RAMDISK0":
[name]          [inode_num]
-----
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.

```

图 13 重在执行测试

本地调试 git 记录:

```
commit be98c02188059aa051f0326f04eea2fb469aca03 (HEAD -> lab4_challenge2_exec, origin/lab4_challenge2_exec)
Author: gabriel <3481963967@qq.com>
Date: Mon Mar 2 16:04:21 2026 +0800

    my work on lab4_challenge2_noELF is done.

commit 8e180903ccb0c543f0d31ff0a38a8a8820916f8c
Merge: 7e87b0a afee75f
Author: gabriel <3481963967@qq.com>
Date: Sun Mar 1 14:57:56 2026 +0800

    continue to work on lab4_challenge2
```

图 14 lab4-2 git log

7.3 实验调试及心得

本次实验实现了操作系统内核中程序动态替换的关键功能。当用户态程序调用 `exec` 系统调用时，内核需要完全抛弃当前进程的代码、数据、堆栈，并从文件系统读取一个新的 ELF 文件建立全新的执行环境。实验通过 `do_exec` 函数，首先打开指定路径的 ELF 文件并校验其头部；接着遍历进程的 `mapped_info`，逐一解除原有代码段、数据段、堆段和栈段的页表映射并释放物理页，保留 `trapframe` 等系统必需的映射；然后重新分配用户栈、重置堆管理器，并依据新 ELF 的程序头表，为每一个 `PT_LOAD` 段分配物理内存、建立虚拟地址映射、设置正确的读/写/执行权限，最后从文件中读入段内容。整个过程将新程序的入口地址写入 `trapframe->epc`，栈顶写入 `trapframe->regs.sp`，使得内核在执行 `sret` 返回用户态时自然跳转到新程序的第一条指令。

在调试过程中，遇到因未彻底清除旧堆段映射而导致新程序地址空间冲突的问题。旧堆段占用的虚拟地址区域没有被完全释放，新 ELF 的代码段被映射到相同地址时触发缺页异常。通过在卸载阶段显式将堆段的 `va` 和 `npages` 清零，并收缩 `total_mapped_region`，成功保证了新段的顺利加载。

通过本次实验，我认识到 `exec` 系统调用在内核中的实现在本质上是进程的重构。它涉及文件系统和虚拟内存管理的紧密协作，要求内核精确控制进程控制块，页表、段记录、堆栈指针和入口地址中任何错误都会导致进程崩溃。实验使我对程序的加载与执行、地址空间的动态重构以及 ELF 文件格式的运用有了更透彻的理解。

实验八 lab4_challenge3 简易 Shell

8.1 实验目的

本实验旨在深入理解操作系统内核中的进程管理与程序执行替换机制，通过实现一个简易命令行解释器 Shell，增强对进程创建、程序加载、父子进程同步以及文件系统读写的掌握。实验要求修改 PKE 内核，使其能够支持 `fork`、`exec`、`wait` 系统调用，并以此为基础构建用户态 Shell，能够从文件中读取预设命令，逐条解析并创建子进程来执行对应的用户程序，最终等待子进程退出后继续处理下一条命令。

这个实验需要理解 `fork` 创建子进程与 `exec` 替换进程映像的详细路径，掌握进程地址空间中代码段、数据段、堆栈的卸载与重建过程，熟练运用虚拟文件系统读取 ELF 文件并进行动态加载，同时学习如何通过 `wait` 协调父子进程的执行顺序，确保命令按序完成。本次实验中，我们通过综合运用进程调度、内存管理、文件系统以及系统调用等底层知识，将先前各项实验整合，提升在操作系统中构建完整用户交互环境与进程控制的能力。

8.2 实验内容与实验记录

Part1.wait 功能实现

在用户函数库 `user_lib.h` 加入 `wait` 函数的声明，并在对应的 `user_lib.c` 文件中加入对应的实现，从而进入系统调用。在系统调用中加入对 `wait` 功能的支持，当传入参数 `pid = -1` 时等待任意一个子进程退出，当 `pid > 0` 时等待进程号为 `pid` 的特定子进程退出。`pid` 为 -1 时，需要进入一个无限循环，不断扫描进程池 `procs` 中的所有进程寻找父进程是当前进程且状态为 `ZOMBIE` 的子进程，一旦找到需记录其 `pid`，调用 `free_process` 回收该子进程的资源，然后返回该子进程的 `pid`。如果本轮扫描没有找到任何僵尸子进程，说明还没有子进程退出，于是将当前进程状态设为 `BLOCKED`，并调用 `schedule` 主动让出 CPU，等待将来某个子进程退出时唤醒当前进程。被重新调度执行后，循环继续检查。实现代码如下：

```
while (1){
    for (int i = 0; i < NPROC; i++){
        if (procs[i].parent == current && procs[i].status == ZOMBIE){
            pid = procs[i].pid;
```

```

        free_process(&procs[i]);
        return pid;
    }
}
current->status = BLOCKED;
schedule();
}

```

当 `pid` 不为 -1 时，需要对 `pid` 进行合法性检查。`pid` 合法则通过主动阻塞进入一个忙等循环，只要指定子进程的状态不是 `ZOMBIE`，当前进程就不断将自己设为 `BLOCKED` 并调用 `schedule` 切换，让出 CPU。每次被重新调度执行时，检查该子进程是否仍然有效，如果该子进程的父进程不再是当前进程或该子进程状态变成了 `FREE`，均失效。一旦子进程变为 `ZOMBIE`，退出循环，调用 `free_process` 回收它，最后返回子进程的 `pid`。实现代码如下：

```

if (pid < 0 || pid >= NPROC || procs[pid].parent != current){
    return -1;
}

while (procs[pid].status != ZOMBIE){
    current->status = BLOCKED;
    schedule();
    if (procs[pid].parent != current || procs[pid].status == FREE){
        return -1;
    }
}

free_process(&procs[pid]);
return pid;

```

同时，需要修改释放进程逻辑。由于在相应子进程退出时需要结束父进程的等待状态，需要补全进程退出函数 `sys_user_exit`。当等待的子进程退出时，需要检查其父进程是否处于被阻塞的 `BLOCKED` 状态，若是则需要修改父进程状态为 `READY` 并插入就绪队列即可。实现代码如下：

```

if (current->parent && current->parent->status == BLOCKED){
    current->parent->status = READY;
    insert_to_ready_queue(current->parent);
}

```

补充 `do_fork` 函数，由于基础实验仅实现了代码段的复制，我们需要继续实现数据段的复制并保证 `fork` 后父子进程的数据段相互独立。子进程继承父进程

的虚拟地址空间布局，并且对数据段每个页进行完整复制，保证数据隔离性，实现代码如下：

```
case DATA_SEGMENT:
    uint64 data_va = parent->mapped_info[i].va;
    for (int pg = 0; pg < parent->mapped_info[i].npages; pg++)
    {
        uint64 page_va = data_va + pg * PGSIZE;
        uint64 page_pa = lookup_pa(parent->pagetable, page_va);
        void *new_pa = alloc_page();
        memcpy(new_pa, (void *)page_pa, PGSIZE);
        user_vm_map((pagetable_t)child->pagetable, page_va, PGSIZE,
                    (uint64)new_pa, prot_to_type(PROT_WRITE | PROT_READ, 1));
    }

    child->mapped_info[child->total_mapped_region].va = parent->mapped_info[i].va;
    child->mapped_info[child->total_mapped_region].npages =
        parent->mapped_info[i].npages;
    child->mapped_info[child->total_mapped_region].seg_type = DATA_SEGMENT;
    child->total_mapped_region++;
    break;
```

Part2.exec 功能合并

exec 重载执行不再只接受固定一个参数，参数数量任意，因此需要修改函数声明为 `int exec(const char *pathname, const char *argv);` 其余进入系统调用以及系统调用将虚拟地址转换为物理地址再交由 `do_exec` 函数处理的逻辑与上述实验七一致。不同的是本次实验需要通过虚拟文件系统读取程序映像，因此 `do_exec` 函数在 `elf.c` 文件中重新实现。

在执行新程序前需要清理进程，将清理进程函数 `clr_proc(process *proc);` 封装在进程管理文件中，`clr_proc` 函数通过重新分配所有顶层资源包括页表、栈、`trapframe`、段记录、文件表，将进程回退到与刚创建进程时相同的干净状态，实现为 `exec` 加载新程序做好了准备。

在 `elf.c` 文件中实现 `do_exec(process *p, char *path, char *argv);` 函数。首先调用清理进程函数 `clr_proc` 彻底清除目标进程 `p` 的旧地址空间，然后加载目标 `elf` 文件，即通过 `VFS` 只读打开指定路径的 `ELF` 文件，初始化 `ELF` 加载器，将文件指针和进程指针绑定到 `info` 结构体，`elf_load` 读取 `ELF` 的程序头表，为每个 `PT_LOAD` 段分配物理内存、建立虚拟地址映射，并记录段信息到 `p->mapped_info` 中，这个过程会建立新程序的代码段和数据段。实现代码如下：


```

// elf loading. elf_ctx is defined in kernel/elf.h, used to track the loading process.
elf_ctx elfloader;
// elf_info is defined above, used to tie the elf file and its corresponding process.
elf_info info;

info.f = vfs_open(path, O_RDONLY);
info.p = p;
// IS_ERR_VALUE is a macro defined in spike_interface/spike_hrtf.h
if (IS_ERR_VALUE(info.f))
    panic("Fail on opening the input application program.\n");

// init elfloader context. elf_init() is defined above.
if (elf_init(&elfloader, &info) != EL_OK)
    panic("fail to init elfloader.\n");

// load elf. elf_load() is defined above.
if (elf_load(&elfloader) != EL_OK)
    panic("Fail on loading elf.\n");

```

将 ELF 头部记录的入口地址写入进程的 `epc` 并关闭 ELF 文件。为解决参数传递问题，需要构造参数区。第一个堆页作为 `argv` 指针数组，存储指向各参数字符串的指针。由于实验要求传入参数数量固定为 1，将进程的 `a0` 寄存器设为 1 对应 `argc = 1`，`a1` 寄存器设为第一个堆页的虚拟地址对应 `argv` 数组。第二个堆页将 `argv` 指向的物理页直接映射到第二个堆页的虚拟地址上，因为 `argv` 数据在调用 `exec` 之前已经准备好。最后，在指向的第一个堆页中写入第二个堆页的虚拟地址，完成 `argv` 数组的构建。实现代码如下：

```

uint64 va = p->user_heap.heap_top;
void *pa = alloc_page();
p->user_heap.heap_top += PGSIZE;
p->mapped_info[HEAP_SEGMENT].npages++;
user_vm_map((pagetable_t)p->pagetable, va, PGSIZE, (uint64)pa,
    prot_to_type(PROT_WRITE | PROT_READ, 1));

p->trapframe->regs.a0 = 1;
p->trapframe->regs.a1 = va;
va = p->user_heap.heap_top;
p->user_heap.heap_top += PGSIZE;
p->mapped_info[HEAP_SEGMENT].npages++;
user_vm_map((pagetable_t)p->pagetable, va, PGSIZE, (uint64)argv,
    prot_to_type(PROT_WRITE | PROT_READ, 1));
*((char **)pa) = (char *)va;

```


Part3.实验测试结果

测试程序从位于 `hostfs_root` 下的 `shellrc` 文件中读取若干条命令，对于每条命令，我们都会 `fork` 一个新的进程，并 `exec` 对应的程序来完成命令。`exec` 将用第一个参数对应的程序来替换 `fork` 出来的新进程，并将第二个参数的内容传入新的进程。测试提供了五种简化版的命令，分别是 `ls`, `mkdir`, `touch`, `echo`, `cat`。测试结果与实际功能效果一致，部分测试结果如下：

```
User application is loading.
Application: /bin/app_shell
CODE_SEGMENT added at mapped info offset:4
DATA_SEGMENT added at mapped info offset:5
Application program entry point (virtual address): 0x00000000000100b0
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== Shell Start =====

Next command: /bin/app_mkdir /RAMDISK0/sub_dir

=====Command Start=====

User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087f0e000, user stack 0x000000007ffff000, user kstack 0x0000000087f0c000
FS: created a file management struct for a process.
in alloc_proc. build proc_file_management successfully.
do_fork map code segment at pa:0000000087f13000 of parent to child at va:0000000000010000.
going to insert process 1 to ready queue.
going to schedule process 1 to run.
Application: /bin/app_mkdir
CODE_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x0000000000010078

===== mkdir command =====
mkdir: /RAMDISK0/sub_dir
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

Next command: /bin/app_touch /RAMDISK0/sub_dir/ramfile1
```

图 15 简易 shell 测试

本地调试 git 记录：

```
commit a81e44b6206beaa2b7edb80f9eb9076ec9b3378c (HEAD -> lab4_challenge3_shell, origin/lab4_challenge3_shell)
Author: gabriel <3481963967@qq.com>
Date: Tue Mar 3 14:01:03 2026 +0800

    my work on lab4_challenge3 is done.

commit 2865aaab5b068fd2779b051a47c2d1af91ddea68
Author: gabriel <3481963967@qq.com>
Date: Mon Mar 2 18:42:14 2026 +0800

    lab4_challenge3_wait

commit b24df9e1599047f655c4317a2abad3d6df154a07
Merge: 735dbaa afee75f
Author: gabriel <3481963967@qq.com>
Date: Mon Mar 2 16:05:43 2026 +0800

    continue to work on lab4_challenge3
```

图 16 lab4-3 git log

8.3 实验调试及心得

实验构建了操作系统内核中进程协作与命令解释的完整能力。当用户态 Shell 程序从 `shellrc` 文件中逐行读取预设命令后，需要通过 `fork` 创建子进程，再通过 `exec` 将子进程替换为目标命令程序，最后父进程调用 `wait` 等待子进程退出并继续处理下一条命令。这一过程中，内核需要完成进程地址空间的彻底重置、新 ELF 文件的加载与映射、父子进程的同步等待一系列底层操作。实验中，通过 `clr_proc` 将子进程格式化为空白状态，再借助 `elf_load` 从文件系统逐段加载新程序的代码与数据，同时在堆上构建 `argc` 和 `argv` 参数结构，使得新程序的 `main` 函数能够正确接收命令行参数。当命令执行完毕后，子进程调用 `exit` 变为僵尸状态并唤醒可能阻塞的父进程，父进程则通过 `wait` 回收子进程资源，确保了多次命令执行的串行正确性。

在调试过程中，遇到子进程执行 `exec` 后参数传递失败的问题，新程序的 `main` 函数接收到的 `argv` 为空指针。通过追踪发现，在 `do_exec` 中为参数字符串分配的物理页虽然映射到了堆区，但对应的 `argv` 指针数组中填入的虚拟地址在 `clr_proc` 重置页表后已不再有效。解决方法是确保 `clr_proc` 建立全新的页表后，再将 `argv` 数据所在的物理页映射到堆上预留的虚拟地址，并正确设置 `a0`、`a1` 寄存器。

通过本次实验，我深刻体会到一个简易 Shell 需要进程管理、内存管理、文件系统的精密协作。本次实验设计命令解析、子进程创建、程序映像替换、参数传递、进程退出、父进程回收，其中涉及前面实验中的设计方法与思路。实验使我对进程生命周期有了更完整的理解，尤其是 `fork` 与 `exec` 分离的设计思想。`fork` 负责复制执行环境，`exec` 负责替换执行内容，`wait` 负责资源的最终回收，三者共同构成了操作系统中创建到执行再到回收的完整进程管理流程。构建 `argc`、`argv` 的过程让我深入理解了用户程序 `main` 函数参数的真正来源，也加深了对 RISC-V 函数调用约定和栈布局的认识。这些底层知识为理解现代操作系统中的程序加载器、命令行解释器等实现原理提供了实践经验。

实验九 challengeX : Put it all together

9.1 实验目的

本实验为综合实验，给定的应用为前面四个实验的所有挑战实验，最终实现一个支持交互式 Shell 的稳定操作系统环境。具体实现目标如下：

设计操作系统界面，包括命令行提示符、操作系统启动提示、操作系统退出提示、操作系统异常提示，实现用户交互流畅；

将全部实验 lab1-4 的挑战实验 Challenge1-3 融入 Shell，每个挑战对应的程序作为 app，可在简易 shell 中调用执行。全部 app 预先编译、链接好，放在 host 端的 bin 子目录下，通过 PKE 的 VFS 对它们进行访问；

实现系统同时对脚本与命令行的支持。在 lab4_challenge3 中已经实现对脚本文件的支持，本需要设计命令行，支持常用命令，并支持上述给定 app 能够通过命令行调用执行；

正确处理异常情况，实现内核捕获到应用程序的严重错误时内核不会崩溃。在触发异常时操作系统能够正确处理错误行为并报告当前错误信息，同时操作系统能够正确退出异常状态，回到出错子进程的父进程，保障内核稳定性。

9.2 实验内容与实验记录

Part1.编写 Makefile 文件

将所有 app 功能整合到项目文件后，为支持已有挑战实验功能，需要编写 Makefile 文件，将全部 app 按照规则预先编译、链接好，并将其输出到宿主机文件系统的/bin 目录下。这里以挑战实验 lab4-challenge2 为例，补充代码如下：

```
USER_EXEC_CPPS      := user/app_exec.c user/user_lib.c
USER_EXEC_OBJS      := $(addprefix $(OBJ_DIR)/, $(patsubst %.c,%.o,$(USER_EXEC_CPPS)))
USER_EXEC_TARGET    := $(HOSTFS_ROOT)/bin/app_exec

@-mkdir -p $(dir $(USER_EXEC_OBJS))
$(USER_EXEC_TARGET): $(OBJ_DIR) $(UTIL_LIB) $(USER_EXEC_OBJS)
    @echo "linking" $@ ...
    -@mkdir -p $(HOSTFS_ROOT)/bin
    @$$(COMPILE) --entry=main $(USER_EXEC_OBJS) $(UTIL_LIB) -o $@
    @echo "User app has been built into" \"$(@)\"
```

最终将 `app` 融入构建系统，将构建好的 `app` 添加为 `all` 和 `run` 目标的依赖项，从而通过 `make` 将全部 `app` 自动编译生成在 `/bin` 文件夹下。

Part2.exec 改进

在 `clr_proc(p)` 之前将用户提供的参数字符串保存在内核栈上，使其完全不受后续地址空间清空的影响，实现代码如下：

```
char argv_buf[MAX_ARGV_LEN];
if (argv)
{
    size_t len = 0;
    while (len < MAX_ARGV_LEN - 1 && ((char *)argv)[len] != '\0')
        len++;
    len++;
    if (len > MAX_ARGV_LEN)
        len = MAX_ARGV_LEN;
    memcpy(argv_buf, (void *)argv, len);
    argv_buf[MAX_ARGV_LEN - 1] = '\0';
}
else
{
    argv_buf[0] = '\0';
}
```

清空旧进程后，从内核缓冲区写入新堆页，保证不会因为原参数字符串所在的区域被释放或复用导致，实现代码如下：

```
uint64 va_str = p->user_heap.heap_top;
void *pa_str = alloc_page();
memset(pa_str, 0, PGSIZE);
memcpy(pa_str, argv_buf, MAX_ARGV_LEN);
user_vm_map((pagetable_t)p->pagetable, va_str, PGSIZE, (uint64)pa_str, prot_to_type(PROT_WRITE | PROT_READ, 1));
*((char **)pa) = (char *)va_str;
```

Part3.物理地址转换及文件 IO 方式改变

由于在 `lab1` 中采用直接映射的方式，即物理地址和虚拟地址直接对应，而在 `lab2` 开始采用 `Sv39` 的分页机制，需要将 `lab1` 中利用物理地址完成回溯追踪的函数进行修改。`sys_user_print_backtrace` 函数中只有在读取栈内容从而获得函数入口地址的过程中需要获取物理地址，因此只需要在每次读取栈前进行虚拟地址

到物理地址的转换即可，代码如下：

```
uint64 user_fp = tf->regs.s0;
if (user_fp == 0) return 0;

void *pa = user_va_to_pa(pagetable, (void *)(user_fp - 8));
if (!pa) return 0;
uint64 user_fp_next = *(uint64 *)pa;

for (int i = 0; i < depth; i++){
    if (user_fp_next == 0) break;
    pa = user_va_to_pa(pagetable, (void *)(user_fp_next - 8));
    if (!pa) break;
    uint64 return_address = *(uint64 *)pa;
    char *name = find_func_name(return_address);
    sprintf("%s\n", name);
    if (strcmp(name, "main") == 0) break;

    pa = user_va_to_pa(pagetable, (void *)(user_fp_next - 16));
    if (!pa) break;
    user_fp_next = *(uint64 *)pa;
}
```

同时将虚拟地址转换为源程序中的符号即根据地址获取函数名的函数需要做调整，由于在 lab4 之前文件的 IO 是采用使用 HTIF 直接接口 `spike_file_open`、`spike_file_pread`、`spike_file_close`，在最终版使用 VFS 接口 `vfs_open`、`vfs_read`、`vfs_lseek`、`vfs_close`，保证读取 ELF 文件时经过了 PKE 的完整虚拟文件系统层，可以统一访问 `hostfs` 和 `RFS` 中的文件。

Part4.实现双核系统

除在实验二和实验四中对支持双核系统做出的修改外，需要支持在开启 `-p2` 后能够实现后续传入参数的多个应用能够分别绑核，在 `load_user_program` 中进行修改即可，实现能够根据核号 `tp` 动态选择应用，代码如下：

```
if (!argc) panic("You need to specify the application program!\n");
char *app_path = (tp < argc) ? arg_bug_msg.argv[tp] : arg_bug_msg.argv[0];
sprintf("hartid = %d: Application: %s\n", tp, app_path);
load_bincode_from_host_elf(proc, app_path);
sprintf("hartid = %d: Application program entry point (virtual address): 0x%lx\n", tp, proc->trapframe->epc);
```

同时，需要修改进程调度文件 `sched`，保证在双核状态下可以并行地调用 `schedule` 去选择并执行不同的进程，且两个核不会相互影响，保证双核独立性。

在操作就绪队列时，使用自旋锁进行保护，通过 `spinlock_amo_lock/unlock` 确保在多核同时对队列插入和删除时的原子性，避免链表损坏。在就绪队列为空时，采用循环保证系统不会崩溃，如果没进程可运行则执行 `asm volatile("wfi")` 让出 CPU，等待下一次时钟中断或其他事件后自动重试；只有在所有核心上的全部进程都退出了，才由 0 号核执行 `shutdown` 关闭系统，非 0 核则循环等待关机。实现关键代码如下：

1. 封装锁初始化系统全局退出判断函数

```
static inline void rq_init_once(){
    if (!rq_lock_initied)
    {
        spinlock_amo_init(&rq_lock);
        rq_lock_initied = 1;
    }
}

static inline int all_process_done(){
    for (int i = 0; i < NPROC; i++)
        if (procs[i].status != FREE && procs[i].status != ZOMBIE)
            return 0;
    return 1;
}
```

2. 就绪队列入队并发保护

```
spinlock_amo_lock(&rq_lock);

if (ready_queue_head == NULL){
    proc->status = READY;
    proc->queue_next = NULL;
    ready_queue_head = proc;
    spinlock_amo_unlock(&rq_lock);
    return;
}

process *p;
for (p = ready_queue_head; p->queue_next != NULL; p = p->queue_next){
    if (p == proc){
        spinlock_amo_unlock(&rq_lock);
        return; // already in queue
    }
}

if (p == proc){
```

```

spinlock_amo_unlock(&rq_lock);

return;
}

```

```

p->queue_next = proc;
proc->status = READY;
proc->queue_next = NULL;

```

```

spinlock_amo_unlock(&rq_lock);

```

3. 修改进程调度保证正确退出

```

while (1){
    // 1) 先尝试从就绪队列取任务
    spinlock_amo_lock(&rq_lock);
    if (ready_queue_head){
        current[tp] = ready_queue_head;
        ready_queue_head = ready_queue_head->queue_next;
        current[tp]->queue_next = NULL;
        if (current[tp]->status != READY){
            sprintf("warn: schedule got non-READY proc pid=%d status=%d on hart %d\n",
                    current[tp]->pid, current[tp]->status, tp);
        }
        current[tp]->status = RUNNING;
        spinlock_amo_unlock(&rq_lock);
        switch_to(current[tp]);
        return;
    }
    spinlock_amo_unlock(&rq_lock);

    // 2) 队列空，检查是否全系统结束
    if (all_process_done()){
        if (tp == 0){
            shutdown(0);
        }
        // 非 0 核等待关机
        while (1)
            asm volatile("wfi");
    }

    // 3) 暂无可运行任务，等待中断再重试，避免忙等
    asm volatile("wfi");
}

```


Part5.异常退出处理

系统程序异常退出后，系统不应该崩溃，而是杀死异常进程后回到父进程，因此需要将 `panic` 改为杀死异常进程函数。杀死异常进程函数需要获取当前进程和父进程，然后打印错误信息并释放当前进程，唤醒可能正在等待的父进程，最终重新调度。实现代码如下：

```
void kill_current_process(const char *reason, int code)
{
    int tp = read_tp();
    process *self = current[tp];
    process *parent = self->parent;
    sprint("%s\n", reason);
    sprint("hartid = %d: User exit with code:%d.\n", tp, -1);
    free_process(self);
    if (parent && parent->status == BLOCKED)
    {
        parent->status = READY;
        insert_to_ready_queue(parent);
    }
    schedule();
}
```

Part6.交互界面设计

在确保系统正常运行后，删除所有调试信息。为便于对全部功能进行测试，将全部功能写在 `shellrc` 文件中，当输入 `test` 时即可进行自动进行全部测试。为保证操作系统便于交互，设置启动提示、退出提示、命令行提示符`$`。同时，系统支持常见 linux 命令，包括 `pwd`、`cd`、`cat`、`echo`、`ls`、`mkdir`、`touch`，输入命令 `q` 退出系统，在输入非法命令时，系统不会崩溃而是提示用户输入合法命令并进入接收命令状态。下面以 `pwd` 为例，命令行识别代码：

```
// pwd
if (cmd[0] == 'p' && cmd[1] == 'w' && cmd[2] == 'd' && cmd[3] == 0)
{
    char path[30];
    read_cwd(path);
    printu("cwd:%s\n", path);
    continue;
}
```

Part7.实验测试结果

启动测试，正确初始化并启动操作系统，输出启动提示语、命令行提示符：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike -p2 obj/riscv-pke /bin/start
HTIF is available!
(Emulated) memory size: 2048 MB
In m_start, hartid:0
hartid = 0: Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080268000, PKE kernel size: 0x000000000268000 .
free physical memory address: [0x0000000080268000, 0x0000000087ffffff]
kernel memory manager is initializing ...
In m_start, hartid:1
hartid = 1: Enter supervisor mode...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x000000008000b000
kernel page table is on
kernel page table is on
RAMDISK0: base address of RAMDISK0 is: 0x0000000087f35000
RFS: format RAMDISK0 done!
Switch to user mode...
Switch to user mode...
User application is loading.

=====START=====

$
```

图 17 启动测试

退出测试，正确识别提出命令 q 并关闭操作系统：

```
$ q
Exit.
System is shutting down with exit code 0.
gabriel@LAPTOP-CT8H0900:~/riscv-pke$
```

图 18 退出测试

非法命令行测试，正确处理非法命令行：

```
$ errorcmd
Error command: errorcmd
```

图 19 非法命令行测试

挑战实验功能测试，完成并正确执行全部挑战实验功能，部分输出结果如下：

```
va1=0x0000000000402000 page=0x0000000000402000
va2=0x0000000000402064 page=0x0000000000402000
hello, world!!!
=====Command End=====

Next command: /bin/app_print_backtrace NULL

=====Command Start=====

back trace the user app in the following:
f8
f7
f6
f5
f4
f3
f2
=====Command End=====

Next command: /bin/app_relativepath NULL

=====Command Start=====
```

图 20 挑战实验功能测试

异常代码测试，系统正确识别异常命令并定位到异常代码位置，系统正确杀死异常程序并回到父进程：

```
$ test2
Going to hack the system by running privilege instructions.
Runtime error at user/app_errorline.c:14
    asm volatile("csrw sscratch, 0");
Illegal instruction!

hartid = 1: User exit with code:-1.
$
```

图 21 异常测试

```
$ test3
this address is not available!
hartid = 0: User exit with code:-1.
$
```

图 22 越界访问测试

双核测试，两程序能够并行，且当两程序均需要分配新地址时二者不会分配到同一块物理地址，但虚拟地址可能一致：

```
$ test1
=== user alloc 0 @ vaddr 0x00402000
=== user alloc 0 @ vaddr 0x00403000
=== user alloc 0 @ vaddr 0x00404000
=== user alloc 0 @ vaddr 0x00405000
=== user alloc 0 @ vaddr 0x00406000
=== user 0: 0
=== user 0: 1
>>> user alloc 1 @ vaddr 0x00402000
=== user 0: 2
>>> user alloc 1 @ vaddr 0x00403000
=== user 0: 3
>>> user alloc 1 @ vaddr 0x00404000
=== user 0: 4
>>> user alloc 1 @ vaddr 0x00405000
>>> user alloc 1 @ vaddr 0x00406000
>>> user 1: 5
>>> user 1: 6
>>> user 1: 7
>>> user 1: 8
>>> user 1: 9
$
```

图 23 双核测试 1

```
$ test4
>>> app0 is expected to be executed by hart0
>>> app1 is expected to be executed by hart1
$
```

图 24 双核测试 2

pwd、cd 命令测试：

```
$ pwd
cwd:/
$ cd
>/RAMDISK0
$ pwd
cwd:/RAMDISK0
```

图 25 pwd&cd 测试

ls 命令测试:

```
$ ls
>/RAMDISK0/sub_dir
[name]          [inode_num]
ramfile1        2
ramfile2        3
```

图 26 ls 测试

mkdir 命令测试:

```
$ ls
>/RAMDISK0
[name]          [inode_num]
$ mkdir
>/RAMDISK0/sub_dir
mkdir: /RAMDISK0/sub_dir
$ ls
>/RAMDISK0
[name]          [inode_num]
sub_dir         1
```

图 27 mkdir 测试

touch 命令测试:

```
$ ls
>/RAMDISK0/sub_dir
[name]          [inode_num]
$ touch
>/RAMDISK0/sub_dir/ramfile1
$ touch
>/RAMDISK0/sub_dir/ramfile2
$ ls
>/RAMDISK0/sub_dir
[name]          [inode_num]
ramfile1        2
ramfile2        3
```

图 28 touch 测试

echo、cat 命令测试:

```
$ echo
>/RAMDISK0/sub_dir/ramfile1
write content:
hello world
$ cat
>/RAMDISK0/sub_dir/ramfile1
read content:
hello world
```

图 29 echo&cat 测试

相对路径测试:

```
$ pwd
cwd: /RAMDISK0
$ cd
>..
$ pwd
cwd: /
```

图 30 相对路径测试

本地调试 git 记录。实验以 lab4-challenge3 简易 shell 为基础，逐步合成 lab4、lab3、lab2、lab1 实验功能，并逐步修改 exec 复用已释放区域问题、文件 IO 方式仍采用 HTIF 接口问题、系统调用中虚拟地址未转换为物理地址问题，然后修改进程调度、程序文件载入等部分实现双核操作系统，设计 start.c 操作系统启动程序，修复异常程序导致系统崩溃问题，修复交互界面存在大量调试信息问题，修复交互界面缺少命令行提示符等交互信息不规范问题，实现最终的可交互操作系统：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ git log
commit 86ecd9c7ca1eddc10a2ea7f017e0318fa0e49d88 (HEAD -> challengeX, origin/challengeX)
Author: gabriel <3481963967@qq.com>
Date: Tue Mar 10 14:33:35 2026 +0800

    finbase

commit 436f942ef6a062a14c4c8682c0784a76e86a8784
Author: gabriel <3481963967@qq.com>
Date: Tue Mar 10 02:05:22 2026 +0800

    fin3.0

commit 5ce3ca411a07af1171a2dbf7b9a8c7ea5d05f5dd
Author: gabriel <3481963967@qq.com>
Date: Sun Mar 8 20:19:30 2026 +0800

    fin2.0

commit 98941e8af042fb8fd31574ef35259a5ad8b3bcf9
Author: gabriel <3481963967@qq.com>
Date: Sun Mar 8 02:47:46 2026 +0800

    fin1.0

commit 9a9cbe773ef2bc14bdf90aa401f822f90a11b53a
Author: gabriel <3481963967@qq.com>
Date: Sat Mar 7 15:19:54 2026 +0800

    2core4one

commit 73675fbda41c9084ef9c5af04b410689b5e379b8
Author: gabriel <3481963967@qq.com>
Date: Sat Mar 7 13:33:45 2026 +0800

    c1.

commit d49db618fa739063cd9013a1a493e53dea35a24e
Author: gabriel <3481963967@qq.com>
Date: Fri Mar 6 02:27:17 2026 +0800

    c2.

commit bca61452ff2c8819afb8c6473460015880b25d08
Author: gabriel <3481963967@qq.com>
Date: Wed Mar 4 15:16:37 2026 +0800

    c3.

commit 539d789d09468495c124586e29917e6aa76735b7
Author: gabriel <3481963967@qq.com>
Date: Wed Mar 4 14:05:55 2026 +0800

    c4.

commit a81e44b6206beaa2b7edb80f9eb9076ec9b3378c (origin/lab4_challenge3_shell, lab4_challenge3_shell)
Author: gabriel <3481963967@qq.com>
Date: Tue Mar 3 14:01:03 2026 +0800

    my work on lab4_challenge3 is done.

commit 2865aaab5b068fd2779b051a47c2d1af91ddea68
Author: gabriel <3481963967@qq.com>
Date: Mon Mar 2 18:42:14 2026 +0800
```

图 31 challengeX git log

9.3 实验调试及心得

Part1.实验原理分析

本次综合实验的核心目标是将 PKE 操作系统从单核、单进程、无文件系统的简单原型，逐步升级为支持多核并发、虚拟内存与页式管理、完整虚拟文件系统以及进程同步与通信的交互式操作系统。实验以可交互式操作系统为最终成果，深度整合了异常捕获与调试信息解析、多核启动与同步、堆内存精细化管理、多核内存隔离与自旋锁、信号量同步机制、写时复制优化以及程序动态替换 `exec` 模块。这些模块并非简单叠加，而是通过统一的系统调用接口、进程控制块、虚拟地址空间与虚拟文件系统框架进行协同。核心过程 `shell` 解析命令后通过 `fork` 创建子进程，子进程通过改进后的 `exec` 加载新程序映像，此过程依赖于 `clr_proc` 安全重置地址空间，并利用页表操作建立新映射；当多个进程并发运行时，多核调度器借助自旋锁保护就绪队列，并通过 `wait` 机制协调父子进程的退出顺序；若应用程序触发非法指令或访问越界，内核不再崩溃，而是借助 `debug_line` 段解析出错误行号，调用 `kill_current_process` 终止异常进程并唤醒阻塞的父进程。整个系统通过分层隔离与同步原语，保证了多核环境下资源访问的正确性与用户交互的流畅性。

Part2.调试过程

实验在 `switch_to` 中需要为进程的 `trapframe` 填充好内核栈指针、内核页表、陷阱处理函数入口以及硬件核 ID 信息，这样当进程将来因系统调用、中断或异常从用户态陷入内核时，汇编级别的入口 `smode_trap_vector` 可以直接从这些预置的字段中获取必要的上下文，快速完成栈切换、页表切换并跳转到 C 语言处理函数，从而高效、安全地进入内核处理流程。

在 `switch_to` 中未添加对核 ID 的填充导致双核测试出现卡死、未知调用号问题，而单核测试能够通过。错误结果如下：

```
$ test1
Unknown syscall 2280816488

System is shutting down with exit code -1.
gabriel@LAPTOP-CT8H0900:~/riscv-pke$
```

图 32 双核 bug

根据错误结果可以知道问题出现在对多核的支持上，因此在多核的流程上插入调试信息，很快定位到最开始的 `switch_to` 中，发现在填充过程中缺少将核编号填入的代码，加入 `proc->trapframe->regs.tp = read_tp();` 填充核编号后，系统能够正常运行：


```

$ test1
=== user alloc 0 @ vaddr 0x00402000
=== user alloc 0 @ vaddr 0x00403000
=== user alloc 0 @ vaddr 0x00404000
=== user alloc 0 @ vaddr 0x00405000
=== user alloc 0 @ vaddr 0x00406000
=== user 0: 0
=== user 0: 1
>>> user alloc 1 @ vaddr 0x00402000
=== user 0: 2
>>> user alloc 1 @ vaddr 0x00403000
=== user 0: 3
>>> user alloc 1 @ vaddr 0x00404000
=== user 0: 4
>>> user alloc 1 @ vaddr 0x00405000
>>> user alloc 1 @ vaddr 0x00406000
>>> user 1: 5
>>> user 1: 6
>>> user 1: 7
>>> user 1: 8
>>> user 1: 9
$

```

图 33 双核 bug 修复

Part3.实验收获与建议

通过本次综合设计，我对操作系统的整体架构有了系统的认识。我明白了项目文件的机制与策略分离的设计原则，无论是虚拟内存的缺页处理，还是进程调度、文件系统挂载，内核提供的是稳定的底层机制，而具体的管理策略可在不同模块中灵活替换。调试过程中，我曾因未正确将用户态虚拟地址转换为物理地址而导致 exec 后参数传递失败，因未同步 TLB 刷新而读到过期数据，因未写入 tp 导致内核崩溃，这些细节让我深刻体会到操作系统代码的严谨性，任何疏忽都可能被并发或边界条件放大为系统崩溃。此外，实验让我熟练掌握了 RISC-V 特权级切换、Sv39 页表格式、原子指令的内存序语义以及 ELF 文件解析等底层知识，真正将课程理论转化为工程实践。本次实验内容丰富，让我更加深刻的学习了操作系统的实现与设计理念。希望能够在项目中提供更丰富的调试支持并添加调试开关，便于更清晰简单的定位错误。